

■ Introduce ontology and domain independent axioms of Event Calculus.

Ontology:

- $\text{HoldsAt}(F, T)$: Fluent F holds at instant T
- $\text{Happens}(E, T)$: Event E happens at T
- $\text{Initiates}(E, F, T)$: Event E causes fluent F to hold at T .
- $\text{Terminates}(E, F, T)$: Event E causes fluent F to cease at T .
- $\text{Clipped}(T1, E, T2)$: Fluent F has ceased between $T1$ and $T2$, $T1 < T2$.
- $\text{Initially}(F)$: Fluent F holds at 0.

Domain-independent axioms:

- Two axioms for defining when a fluent is true.
$$\text{Happens}(E, T1) \wedge \text{Initiates}(E, F, T1) \wedge (T1 < T2) \wedge \neg \text{Clipped}(T1, F, T2) \rightarrow \text{HoldsAt}(F, T2)$$
$$\text{Initially}(F) \wedge \neg \text{Clipped}(0, F, T) \rightarrow \text{HoldsAt}(F, T)$$
- One axiom defines the clipping of an event.
$$\text{Happens}(E, T1) \wedge \text{Terminates}(E, F, T1) \wedge (T1 < T < T2) \rightarrow \text{Clipped}(T1, F, T2).$$

■ Introduce Knowledge graph paradigm and the main differences with reference to the Semantic Web.

Knowledge Graph is a collection of entities and relations between them. It is represented usually in graph format where the entities are the nodes and the relations are the arcs.

The main difference with the Semantic Web proposal is the scope of the problem that is trying to be solved. KG paradigm aims for constructing a structure to represent a specific set of information while SW aims to create a common framework for representing and querying data on web, making it easier for machines to understand and operate with it.

Moreover KG are centralized and close, while SW are decentralized and open by nature since it is an approach on the current web.

- Describe the Prolog "vanilla" meta-interpreter, commenting the meaning of each clause, and providing a short example of a meta-interpreter (Possibly different from the usual Prolog SLD resolution strategy)

A meta-interpreter for a language L is defined as an interpreter for L , but written in Prolog.

If we write a Prolog meta-interpreter for the language Prolog, then we are defining the "vanilla" meta-interpreter.

1. $\text{solve}(\text{true}) :- !.$
2. $\text{solve}((A,B)) :- !, \text{solve}(A), \text{solve}(B).$
3. $\text{solve}(A) :- \text{clause}(A,B), \text{solve}(B).$

The first clause says that if the goal is the atom true, then its proven and stops.

The second clause says that if the goal is a conjunction of two goals, then each goal is evaluated separately, giving priority to the left-most.

The third clause says that if the goal is a simple predicate then we will try to find a clause in the program that unifies with the goal and evaluate the body as a subgoal.

With this meta-interpreter as a template we can define many others with different strategies, for example, change the rule of computation of the SLD resolution strategy to right-most:

```
solve(true) :- !.
```

```
solve((A,B)) :- solve(B), solve(A). % Notice we changed  
solve(A) :- clause(A,B), solve(B). % order here
```

■ Shortly describe the RETE algorithm

The RETE algorithm is pattern matching algorithm for rule-based systems.

The RETE algorithm works by creating a network of nodes, which represents the conditions of a production rule and edges, which represents the relations between the rules.

Rules that does not satisfy the conditions are quickly eliminated.

In short, the algorithm evaluates efficiently the rules by considering only the relevant ones.

The algorithm can be optimized to handle large rule sets and dynamic changes to the working memory

■ Illustrate the meaning of the predicate findall. Illustrate also the use of the predicate by means of an example.

The findall predicate is a built-in predicate in prolog that takes three parameters. The variable that will be evaluated, a goal and a list.

In the list there will be stored the variable evaluated that satisfies the goal. Example:

~~child(jose).~~

child(eduardo). % Eduardo is a child

child(daniel). % Daniel is a child

parent(jose, eduardo). % Jose is parent of eduardo

parent(~~eduardo~~_{jose}, daniel). % Jose is parent of daniel.

if we want to get all the children of jose we could use findall/3.

?- findall(X, (child(X), parent(jose, X)), children).

children = [eduardo, daniel]

Here we define the goal ~~as findall~~ all the ~~children~~ clause where jose is parent and X is a child.

■ Introduce forward reasoning in rule-based systems and highlight the difference with backward reasoning.

Forward reasoning is a reasoning method in which the system starts with a given data or knowledge and applies rules to infer new data or facts.

The steps can be summarized in the following ones

1. Match: Search for rules whose LHS matches with the fact and decide which ones will trigger. In Prolog it is used the RETE algorithm for this.
2. Conflict resolution: Triggered rules are scheduled and possible conflicts solved using structures for scheduling like FIFO or μ salience.
3. Execution: RHS are executed and effects performed, changing the KB if needed.

This process is repeated until no more facts can be inferred.

In the backward reasoning, the process is started from the goal and rules are used to generate the needed facts to prove the goal.

■ Describe the distribution semantics adopted in LPAD, using also a short program to illustrate such semantics.

In LPAD (Logic Programming with Annotated Disjunctions), the head of a clause is extended with disjunctions, each annotated with a probability. Example:

rains : 0.8 ; null : 0.2 :- cloudy.
rains : 0.6 ; null : 0.4 :- windy

Each rule has a probability distribution over its head

Semantics are distributed over the possible worlds,

~~if we know something else~~

If we know cloudy and windy then ...

~~cloudy~~
 0.8×0.6
rains :- cloudy
rains :- windy

null :- cloudy
rains :- windy

0.2×0.6

0.2×0.4
null :- cloudy
null :- windy

rains :- cloudy
null :- windy

0.8×0.4

Describe how negation is tackled in Prolog, what is NAF, what is SLDNF, and the issues related to with NAF over terms containing unbounded variables.

Prolog deals with negation using NAF, Negation As Failure, this is assuming that a goal fails if it cannot be proven, if a goal is not true, then it must be false.

This technique is implemented in the resolution algorithm SLDNF. The SLDNF is basically the SLD resolution algorithm extended with negation as failure.

The algorithm works in the following way:

- If the selected literal is positive, then SLD is applied
- If the selected literal is negative, then try to prove if it is positive. If this fails, then the execution success.

The problem with NAF is that if a variable is unbounded, then a goal with that variable cannot be proven. ~~Example:~~

~~$P(x) :- q(x).$
 $q(x).$~~

~~Calling $\text{not}(p(x))$ would return true since $p(x)$ cannot be proven~~

- Write a prolog predicate `count/2` that, given as input a term representing a predicate, it will return the number of solutions for the goal of proving that input predicate.

`count(p(x), Result) :-`

`findall(set(x, p(x), S),
count_elements(S, Result).`

`count_elements([], 0).`

`count_elements([H|T], S) :-`

`count_elements(T, Ss),`

`S is Ss + 1.`

- Write a meta-interpreter `verbose(Goal)` that is evaluated to true if `Goal` can be proved; moreover, after having solved any sub-goal, the meta-interpreter must print (at the standard output) the sub-goal itself.

`verbose(true).is`

~~print~~

`verbose((A,B)) :- verbose(A), verbose(B)`

`verbose(A) :- clause(A,B), solve verbose(B),`

`print(Solved:A).`

■ Present the notion of the cut "!" operator in the Prolog language, shortly explaining how it works. To this end, show a short example of a predicate containing the cut, and to illustrate its effects.

The cut operator is a predefined predicate for controlling the execution of a program.

It aims to make some ~~decisions~~ choices non backtrackable, improving the efficiency of a program.

It works clearing the backtracking stack once is detected.

$p :- q_1, q_2, !, q_3, q_4.$

Here once we success at q_2 , we want be able to backtrack, if q_3 fails, then the whole predicate will fail. while if q_2 fails, we will be able to backtrack.

$p(x) :- q(x), r(x).$

$q(1).$

$q(2).$

$r(2).$

Yes, $x = 2$

$p(x) :- q(x), !, r(x).$

$q(1).$

$q(2).$

$r(2)$

false

Write a meta-predicate that, given an input a predicate `Pred`, returns in `Num-of-clauses` the number of clauses defining the predicate `Pred` in the current program.

`how-many (A, Num-of-clauses) :-`

`findall (A, clause(A,B), S),`

`length (S, Num-of-clauses).`